Computer Science & Engineering Department

Arhitectura Sistemelor de Calcul

Curs 4
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest



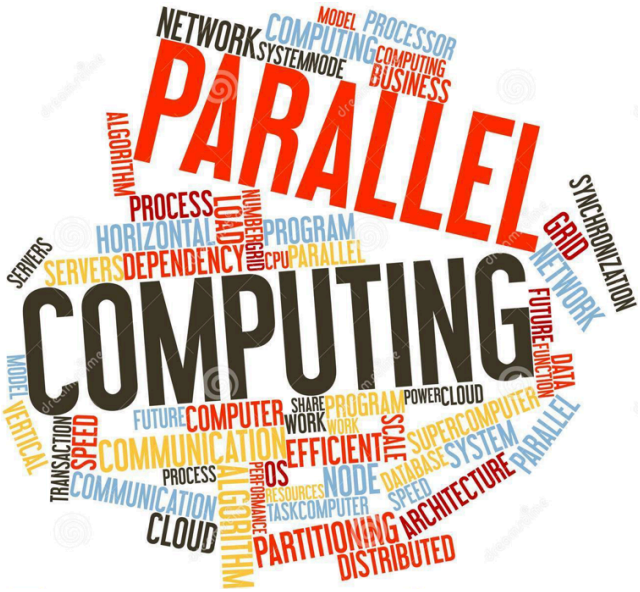
In cursul anterior

- Ierarhia de memorii – Bottleneck-ul SC
- Localitatea datelor
- Cache – design, implementari si exemple
- Imbunatatirea performantelor prin exploatarea ierarhiei de memorie



Cuprins

- De ce avem nevoie de paralelism?
- Structuri de calcul cu prelucrare paralela
- Clasificarea sistemelor de calcul: SISD, SIMD, MISD, MIMD
- Exemple de utilizare a structurilor:
  - SISD
  - SIMD
  - MIMD
- Exemplu cu/fara dependenta de date pe sisteme MIMD

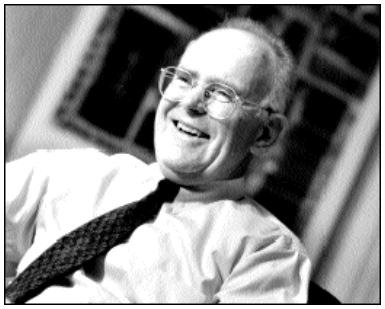


De ce avem nevoie de paralelism?

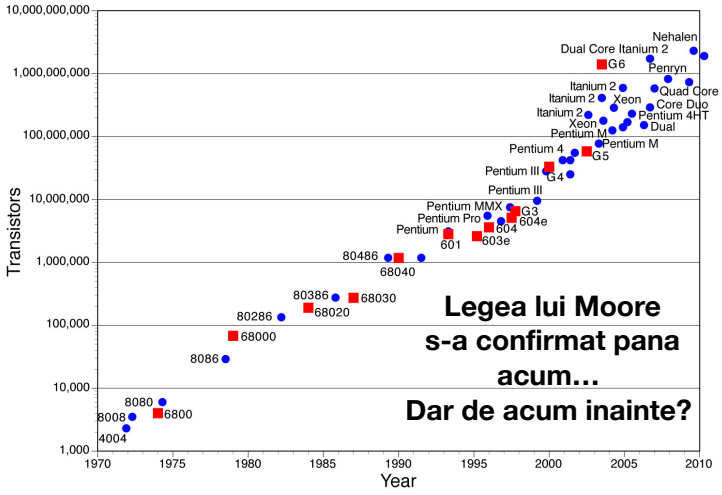
Words of Wisdom

- *"I think there is a world market for maybe five computers."*
  - Thomas Watson, chairman of IBM, 1943.
- *"There is no reason for any individual to have a computer in their home"*
  - Ken Olson, President and founder of Digital Equipment Corporation, 1977.
- *"640KB [of main memory] ought to be enough for anybody."*
  - Bill Gates, Chairman of Microsoft / IBM System Limitation,1981.

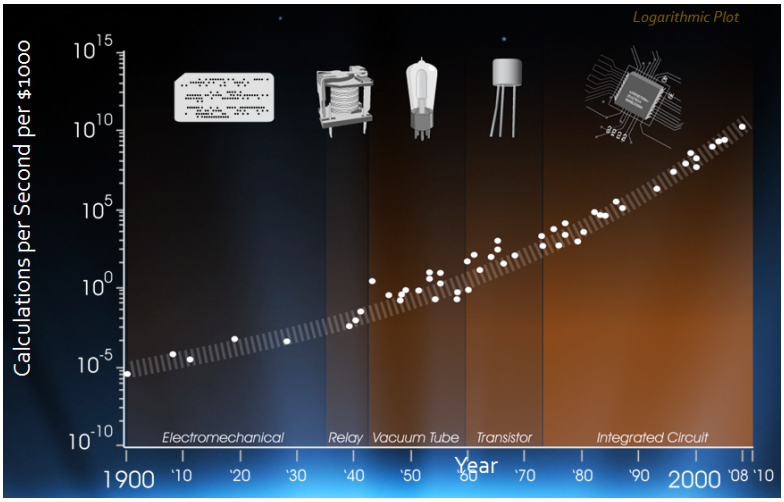
Evolutia Microprocesoarelor

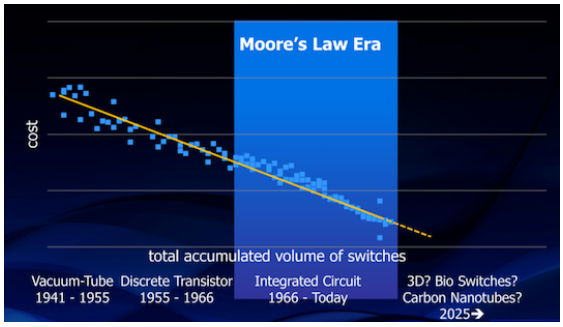


Gordon Moore (cofondator Intel) a prezis in 1965 ca densitatea tranzistoarelor in cipurile cu semiconductori se va **dubla** intr-un interval aproximativ de **18 luni**



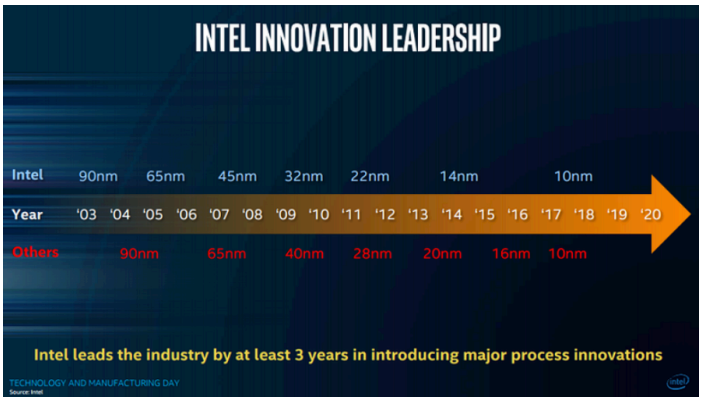
Putere de Calcul vs. Cost



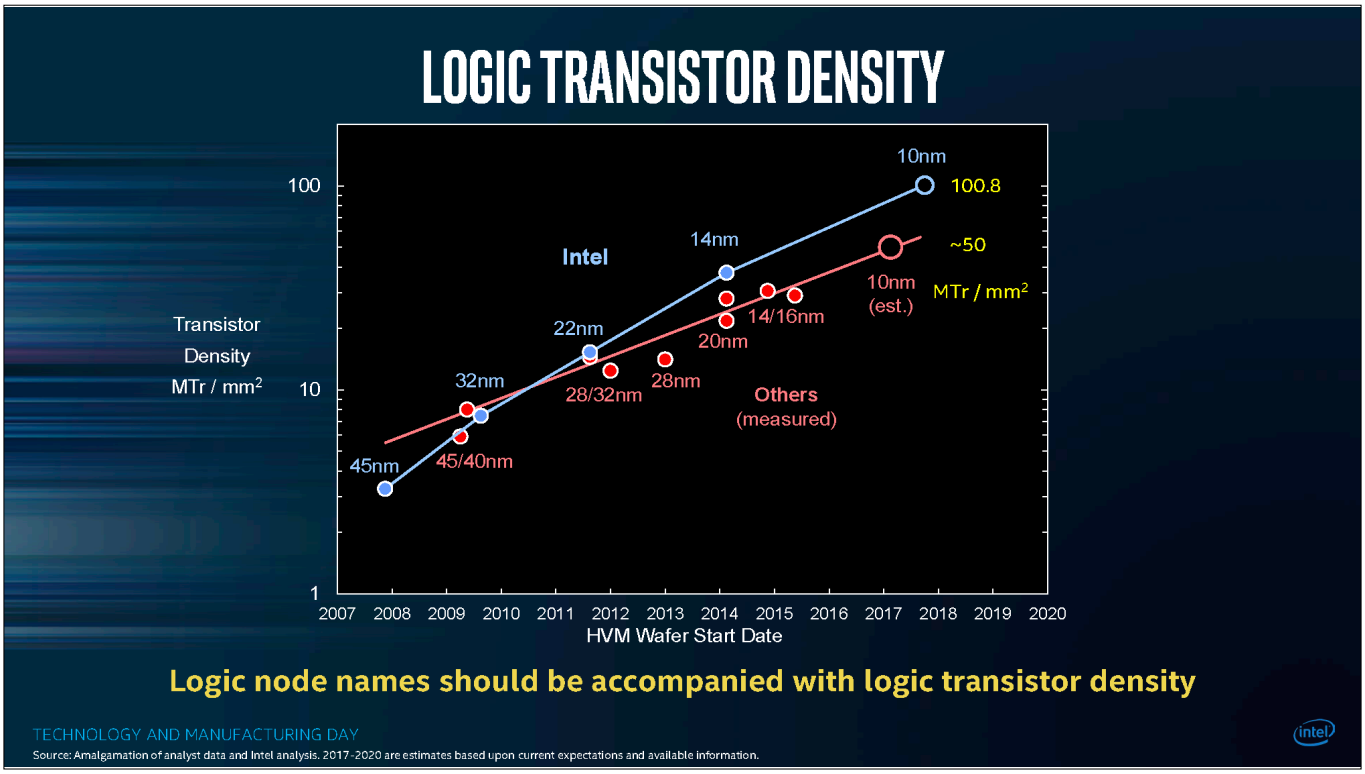


Va putea tehnologia sa continue sa ofere putere de calcul in crestere, la preturi din ce in ce mai mici?

CPU Roadmap vs. Moore's Law

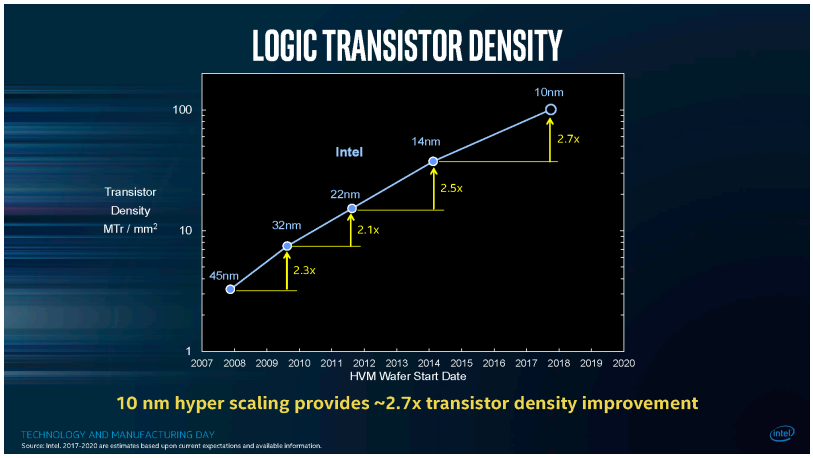


- Up until 1965: 2x / year
- 1965 - 201x: 2x / 2 years
- 201x - 202x: 2x / 3-4 years
- The 2x intervals keep getting longer…

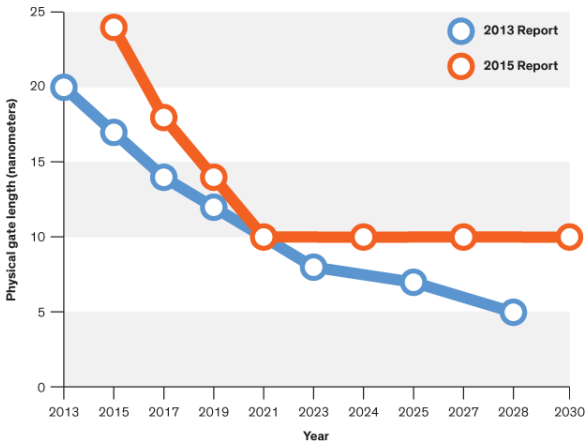


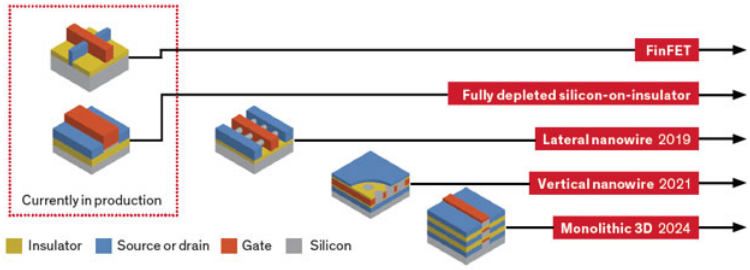
Better Density Improvements than 2x

- Offers possible variants:
  - 14+: better performance
  - 14++: better power usage
- 5nm in production
- 3nm in development
- 1-0.x nm is possible
- Measuring transistor size is not “universal”
  - Can IEEE help?
- What lies after 5nm?

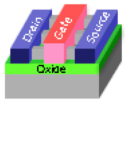
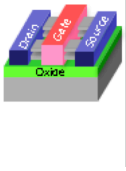
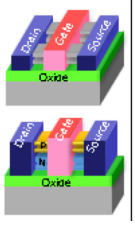
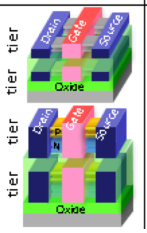
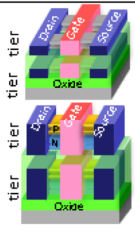
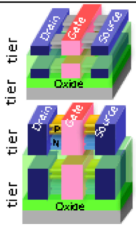


ITRS/IRDS Roadmap 201x

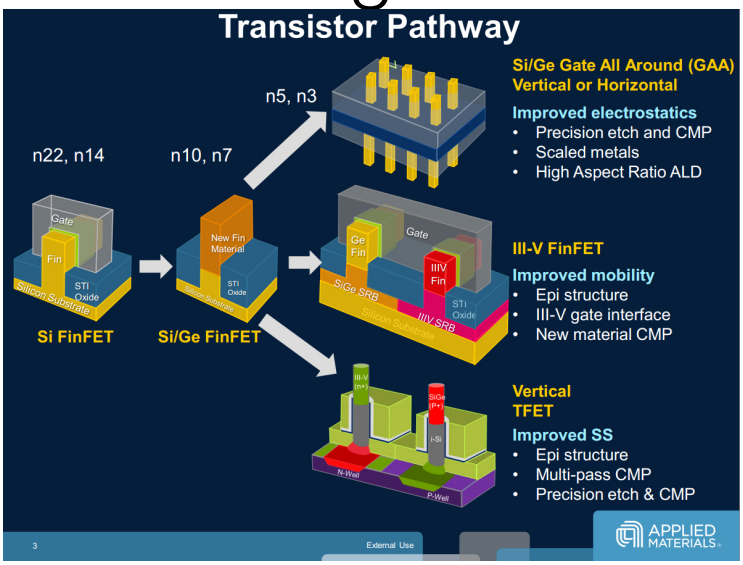




ITRS/IRDS Roadmap 202x - 203x

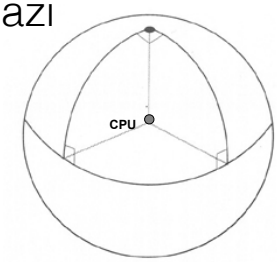
YEAR OF PRODUCTION	2022	2025	2028	2031	2034	2037
	G48M24	G45M20	G42M16	G40M16/T2	G38M16/T4	G38M16/T6
Logic industry "Node Range" Labeling	"3nm"	"2nm"	"1.5nm"	"1.0nm eq"	"0.7nm eq"	"0.5nm eq"
Fine-pitch 3D integration scheme	Stacking	Stacking	Stacking	3DVLSI	3DVLSI	3DVLSI
Logic device structure options	finFET LGAA	LGAA	LGAA CFET-SRAM	LGAA-3D CFET-SRAM	LGAA-3D CFET-SRAM	LGAA-3D CFET-SRAM
Platform device for logic	finFET	LGAA	LGAA CFET-SRAM	LGAA-3D CFET-SRAM-3D	LGAA-3D CFET-SRAM-3D	LGAA-3D CFET-SRAM-3D
						

Posibile Tehnologii la nivel de nm



Un Procesor de 10 TFlops?

- Putem construi un CPU neconcurent (serial) care
  - Ofera 10,000 de miliarde de operatii in virgula mobila pe secunda (10 TFlops)?
  - Opereaza pe 10,000 miliarde de bytes (10 TByte)?
- Este reprezentativ pentru necesitatile din ziua de azi
- Ceasul procesorului trebuie sa fie de 10,000 GHz
- Presupunem ca datele circula cu viteza luminii
- Presupunem ca procesorul este o sfera “ideala”

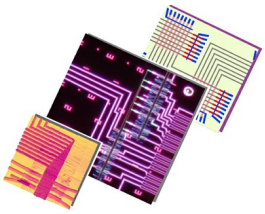


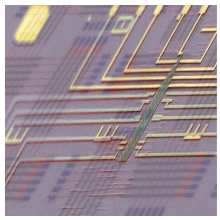
Un Procesor de 10 TFlops?

- Masina genereaza o instructiune pe ciclu, si de aceeas ceasul trebuie sa fie de aproximativ 10,000 GHz ~ 10^{13} Hz
- Datele au de parcurs o distanta intre memorie si CPU
- Fiecare instructiune necesita cel putin 8 bytes de memorie
- Presupunem ca datele circula cu viteza luminii c = $3e^{8}$ m/s
- Astfel, distanta intre memorie si CPU trebuie sa fie r < c / 10^{13} ~ $3e^{-6}$ m
- Apoi, trebuie sa avem 10^{13} bytes de memorie intr-un volum de $4/3\pi r^3 = 3.7e^{-17}$ m^3
- Astfel, fiecare cuvant de memorie trebuie sa ocupe maxim $3.7e^{-30}m^3$ = 3.7 $Angstrom^3$
  - O molecula mica, formata din cativa atomi…
- Densitatea curenta a memoriei este 10GB/cm^3: factor de ordinul 10^{20} sub necesitati!
- Concluzie: Acest procesor nu se poate realiza cu tehnologia actuala.

Nanoprocesoare

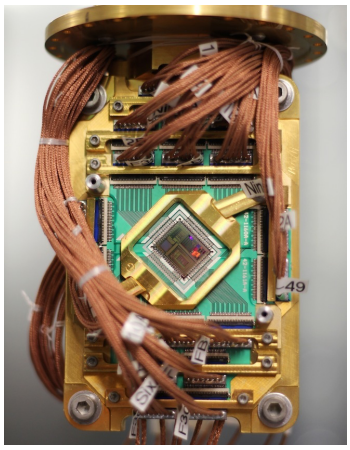
- Harvard University & MITRE Corporation au dezvoltat si demonstrat primul nanoprocesor programabil din lume
  - Circuite la scara nano aranjate ca niste sabloane de nanoprocesoare
  - Firele de interconectare sunt din germaniu-silicon
  - Diametrul total este de doar 30 de nanometri





Quantum Computing

- Quantum bits = Qubits: 0, 1, & 0/1 combinations
  - Superposition - multiple states at the same time
  - Entanglement - particle correlation or “spooky action at a distance”
  - Observation disturbs the system
- Typical examples of applications
  - Factorization: cryptography
  - Simulate quantum systems: new materials & drugs
  - Simultaneous exploration of solution space: optimisation
- Currently machines with just 100s of Qubits:
  - DWave, Google, Microsoft, IBM, NASA, NSA, …



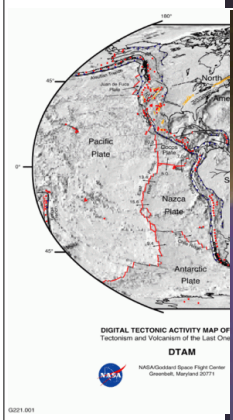


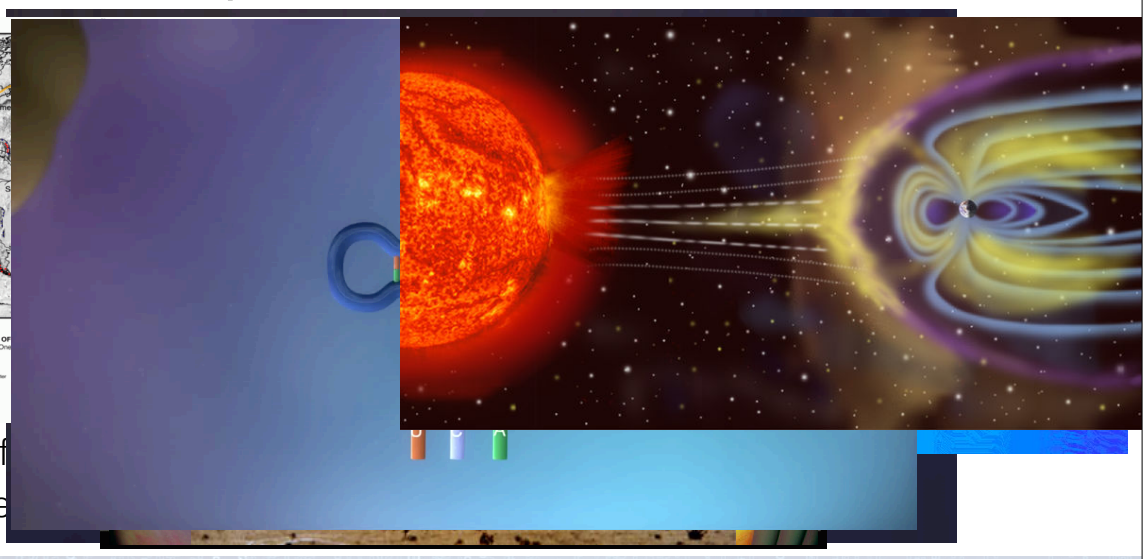
Structuri de Calcul cu Prelucrare Paralela

Paralelismul este Necesar!

- Paralelism la nivel de Bit (Bit-level parallelism)
  - Operatii in virgula mobila
- Paralelism la nivel Instructiune (ILP)
  - Mai multe instructiuni pe ciclu
- Paralelism la nivelul memoriei
  - Suprapunerea intre operatii cu memoria si de calcul
- Paralelism al sistemului de operare
  - Mai multe thread-uri, procese pe CPU-uri multiple, in cadrul aceleasi masini
- Paralelism distribuit
  - Mai multe masini conectate impreuna

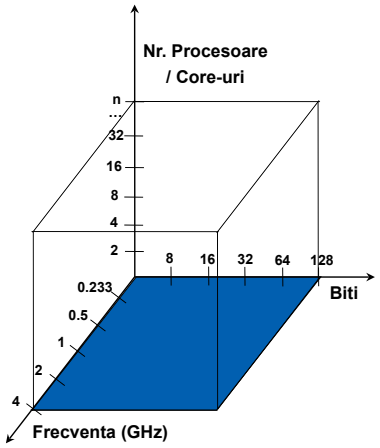
Aplicatii Paralele





- Geof
- Mete

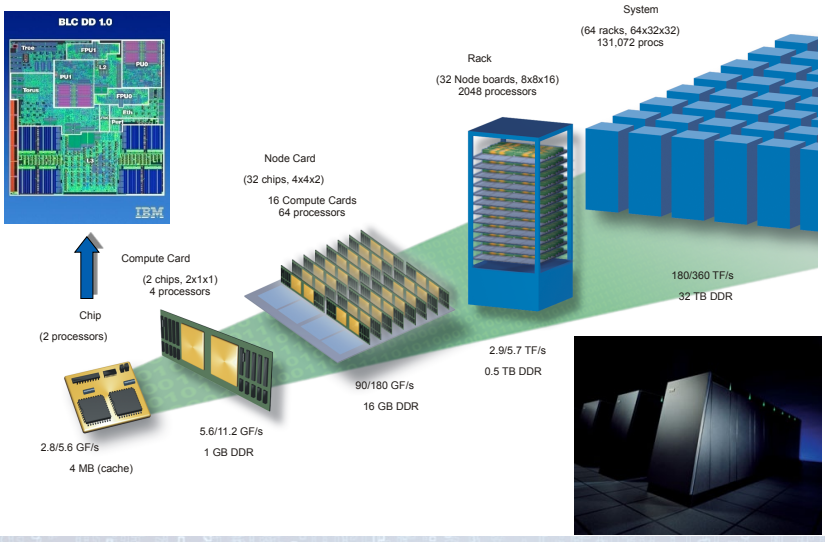
Structuri de Calcul cu Prelucrare Paralela



- Aria → puterea de calcul
- Marimea puterii de calcul: volumul cubului - “n” e limitat “doar” de cost → Arhitecturi paralele de calcul
- Orice problema are un grad de paralelism intrinsec ce depinde de natura ei
- Trebuie tinut seama de:
  - Algoritmi paraleli specifici
  - Limbaje adecvate de programare
  - Limitarile SO
  - Arhitectura intrinseca a SC utilizat

Gradul general de paralelism al codurilor este ~ 10%

IBM BlueGene/L - 131,072 Processors



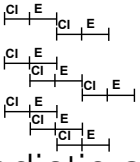
Nivelul Prelucrarii Paralele - CA 23.03

1. Paralelismul la nivel de Bloc/Job:
   - Intre Job-uri diverse
     - Necesare: mecanism de salvare a contextului; un ceas de timp; canale I/O
   - Intre fazele unui Job
     - Citirea sursei programului
     - Compilare
     - Link-are
     - Executarea codului obiect
     - Salvarea rezultatelor
   - Anumite faze ale unui job pot fi suprapuse
2. Paralelismul la nivel de subansamble Hardware:
   - Intre elemente de prelucrare ale vectorilor
   - Intre componentele logice ale dispozitivelor aritmetico/logice (e.g. carry look ahead)

Nivelul Prelucrarii Paralele

3. Paralelismul la nivel de Program:
   - Intre diverse sectiuni (independente) ale unui program
   - Intre buclele (loop-urile) unui program
   - Ambele presupun analiza dependentelor de date!
4. Paralelismul la nivel de Instructiune:
   - Intre diverse faze ale ciclului instructiune:
     - Implementare seriala:
     - Implementare serie-paralela:
     - Implementare paralela:
   - Este necesar un mecanism de predictie al salturilor

CI = Citire & Interpretare
E = Executie

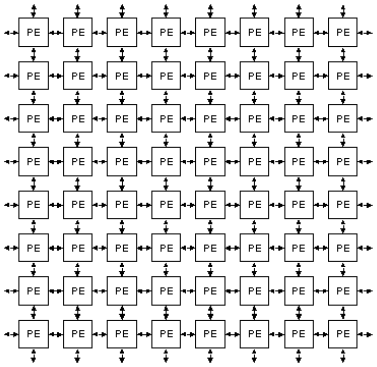




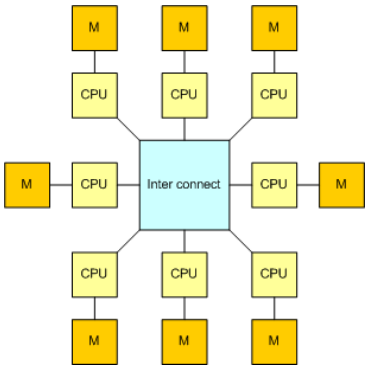
Clasificarea Sistemelor de Calcul

Sisteme Matriceale - Processor Array

- Unitatea de baza a informatiei este vectorul
- Dispun de instructiuni similare SC Von Neumann – operatiile asupra vectorilor sunt efectuate in aceeasi instructiune

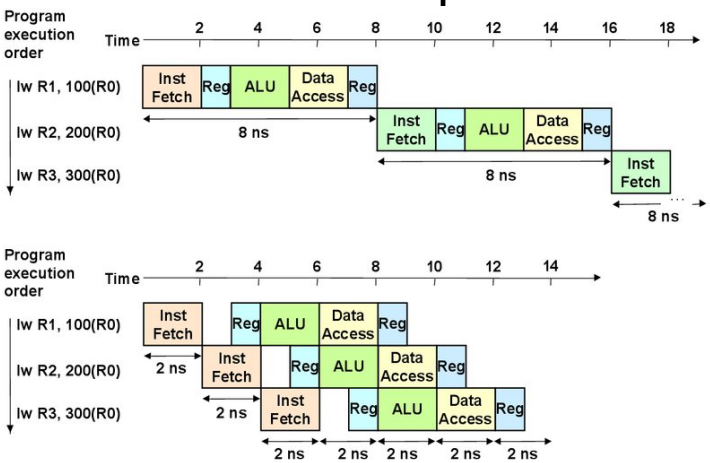


Sisteme Multiprocesor



- Formate din N unitati de prelucrare interconectate printr-o retea de comutare (Strans/Slab cuplata)
- Sistemele lucreaza independent la realizarea aceluiasi Job

Sisteme Pipeline



- Dispun de mai multe unitati de prelucrare asezate in banda de asamblare (RISC):
  - Fiecare UC executa o prelucrare specifica si transfera rezultatul subansamblului adiacent

Taxonomia lui Flynn

- Impartirea sistemelor de calcul in functie de:
  - **Fluxul de Instructiuni** – secventa de instructiuni executate de procesor
  - **Fluxul de Date** – secventa de operanzi manipulata de procesor
- Bazate pe acest criteriu se desprind:

  I. SISD = Single Instruction Single Data Stream (structura Von Neumann)

  II. SIMD = Single Instruction Multiple Data Stream

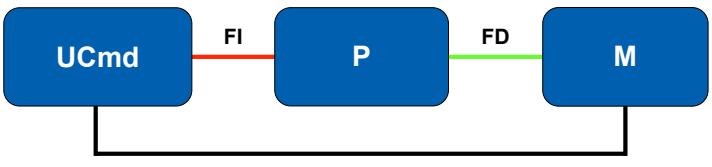
  III.MISD = Multiple Instruction Single Data Stream

  IV.MIMD = Multiple Instruction Multiple Data Stream

I - SISD

- UCmd = Unitate de comanda
- P = Unitate de prelucrare aritmetica
- M = Memorie
- FI = Flux Instructiuni
- FD = Flux Date

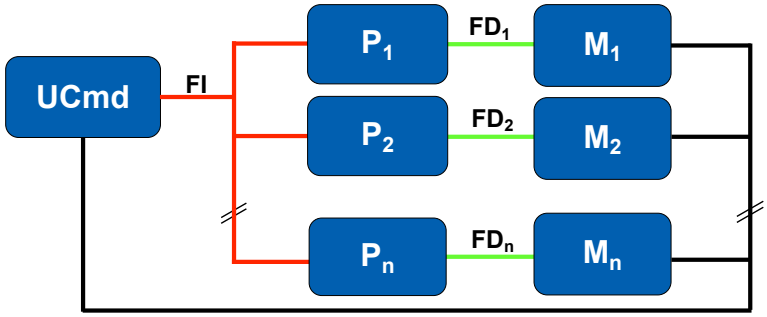
SISD = 1 FI & 1 FD



II - SIMD

- SIMD sunt masini vectoriale:
  - O singura UCmd
  - Mai multe Procesoare si Memorii - orice Procesor vede orice Memorie
- Toate procesoarele fac aceeasi operatie impusa de UCmd prin FI

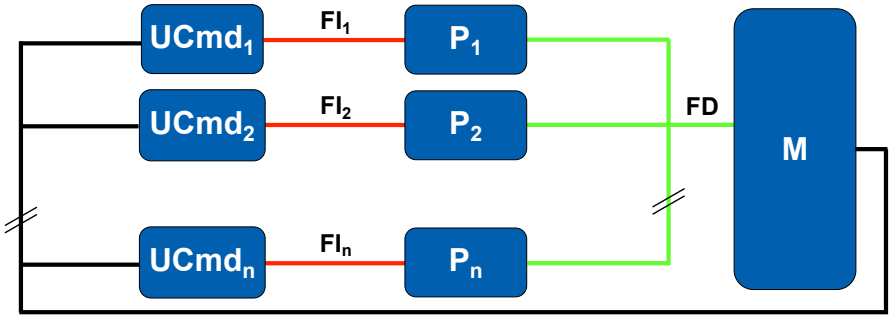
SIMD = 1 FI & n FD



III - MISD

- MISD au:
  - Mai multe UCmd si Procesoare
  - Un singur modul de Memorie
- Domeniu de aplicatie restrans si special: aplicarea altor algoritmi pe aceleasi date (Apps: meteo/evidenta populatiei)

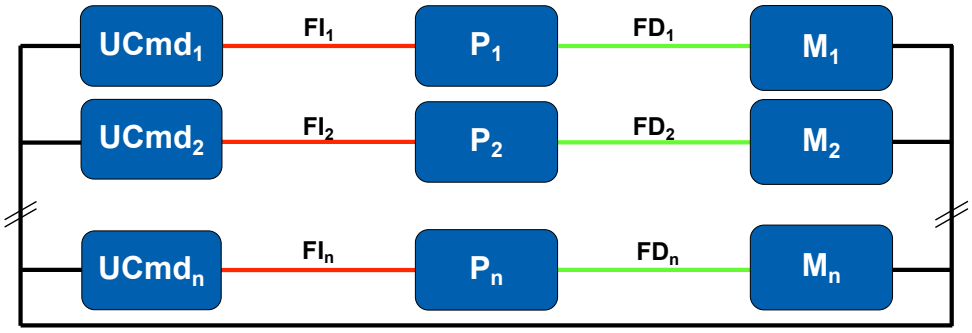
MISD = n FI & 1 FD



IV - MIMD

- MIMD pot comunica: (P-P sau P-M)
- Toate Procesoarele participa la acelasi program
- Mod programare:
  - Shared memory (strans cuplate) – Memorie Partajata (e.g. OpenMP)
  - Distributed memory (slab cuplate) – Transfer de Mesaje (e.g. MPI)

MIMD = n FI & n FD





Exemple de Utilizare a Arhitecturilor

SISD, SIMD, MIMD

Exemplu de Utilizare – SISD

- Problema: A[n, n], B[n, n], C = A x B
- Pe structuri de calcul SISD avem 3 for-uri:

```
for i = 0 to n – 1
  for k = 0 to n – 1
    cik = 0
    for j = 0 to n – 1
      cik = cik + aij * bjk
    end j loop
  end k loop
end i loop
```

- Complexitatea acestui algoritm este… $O(n^3)$! → nesatisfacatoare! (mai ales cand n e mare…)



Exemplu de Utilizare – SIMD

- Aceeasi Problema: A[n, n], B[n, n], C = A x B
- Avem n procesoare & toate executa aceeasi instructiune odata → in fiecare calcul se calculeaza **cate o linie** si **nu doar un element**
- Considerand ($0 \le k \le n - 1$) → se opereaza pentru toti indicii k simultan, adica se calculeaza pe linii:

```
for i = 0 to n – 1
  cik = 0 (0 ≤ k ≤ n – 1)
  for j = 0 to n – 1
    cik = cik + aij * bjk (0 ≤ k ≤ n – 1)
  end j loop
end i loop
```

- Complexitatea acestui algoritm este… $O(n^2)$! → considerabil mai bine ca in cazul SISD



Comentarii si Observatii – SIMD

- Fiecare element al matricei produs C, este o suma ce se efectueaza secvential
- Cele n sume se calculeaza apoi in paralel **!?**
- a_{ij} NU depinde de k:
  - Accesul la aceasta memorie se face **aproximativ secvential**
  - Asadar **nu** e chiar "**sumele se calculeaza in paralel**"!
- Solutia: structurile SIMD **trebuie** sa dispuna de o instructiune de Broadcast si o retea de comutare (RC) pentru acest Broadcast
- P_j citeste a_{ij} prin RC
  - Constanta a_{ij} e difuzata catre toate procesoarele
- Concluzie SIMD: sunt **probleme semnificative** la **comunicatiile** inter-procesoare, la **accesul si organizarea datelor**!



Exemplu de Utilizare – MIMD

- Aceeasi Problema: A[n, n], B[n, n], C = A x B
- O UCmd/P trebuie sa preia functia de master: organizare si control + partajarea calculelor pe procesoare individuale
- Conway a propus o metoda cu 2 primitive: FORK & JOIN
  - FORK: desface un FI in n FI executabile simultan pe procesoare indep
  - JOIN: reuneste n FI intr-unul singur cand cele n FI s-au terminat

```
for k = 0 to n – 2                    // nu si pentru el insusi
   FORK Adr
end k loop
Adr:
   for i = 0 to n – 1
      cik = 0 (0 ≤ k ≤ n – 1) // pe exact k coloane
      for j = 0 to n – 1
         cik = cik + aij * bjk (0 ≤ k ≤ n – 1)
      end j loop
   end i loop
   JOIN
```

Complexitatea este… $O(n^2)$!
→ la fel ca SIMD



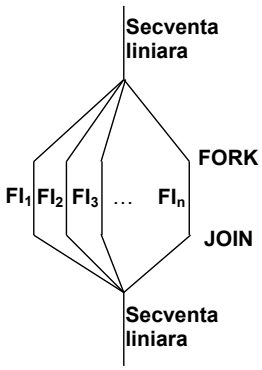
Comentarii si Observatii – MIMD

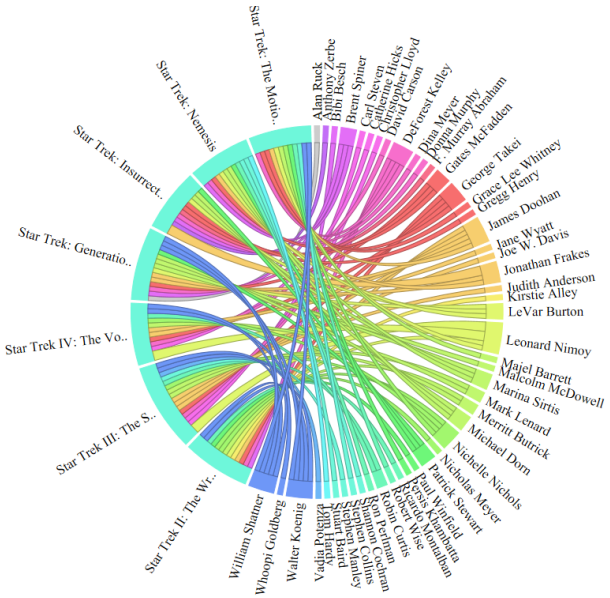
- In mod deliberat programul pt SIMD a fost scris a.i. actiunile P-urilor sa simuleze actiunile din structura MIMD
- Fiecare P_j calculeaza un C_{ik} in paralel
- Diferente SIMD/MIMD:
  - In SIMD procesoarele se sincronizau la nivel de instructiune
  - In MIMD nu exista (neaparat) sincronizari intre FI ale procesoarelor
  - La SIMD se calculeaza elementele lui C pe linie si coloana (FI unic)
  - La MIMD ∀ procesor poate calcula ∀ element din C (FI-uri diferite)
- Castigul e acelasi: se pot folosi mai multe perechi FORK/JOIN
- Concluzie MIMD:
  - Mai usor de programat & utilizat decat SIMD, dar mai scump!

Totul se plateste…

Implementarea FORK

- Instructiunea JOIN n:
  - Pentru fiecare FI exista un contor unic ce e initializat cu 0 la inceput
  - Pentru fiecare FI contorul este incrementat si comparat cu n
  - Trebuie ca toate procesele sa se fi terminat pana la n si abia apoi se continua cu urmatoarea secventa liniara pe P1 (master)





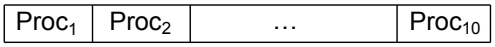
Exemplu cu/fara Dependenta de Date

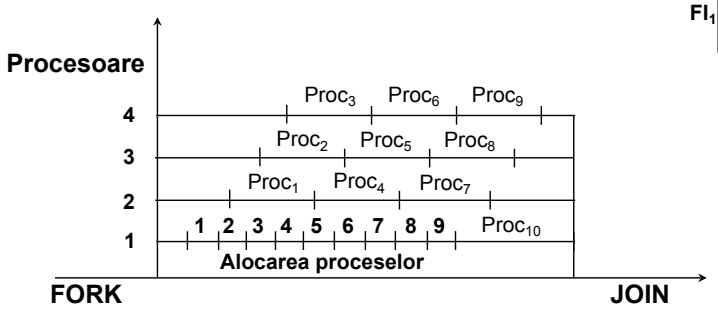
Exemplu fara Dependenta de Date

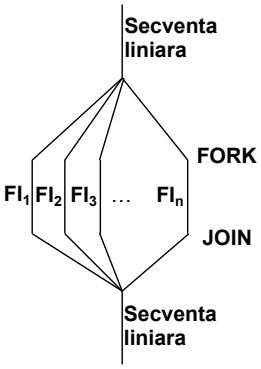
- Patru procesoare P_1, P_2, P_3, P_4 si 10 procese $Proc_i$ (i = 1…10)
- Modelul de executie este:

```
for i = 0 to 9
   FORK Adr
end i loop
Adr:
   Proci
   JOIN
```

Coada de asteptare a proceselor:

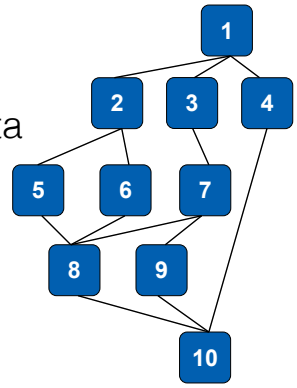


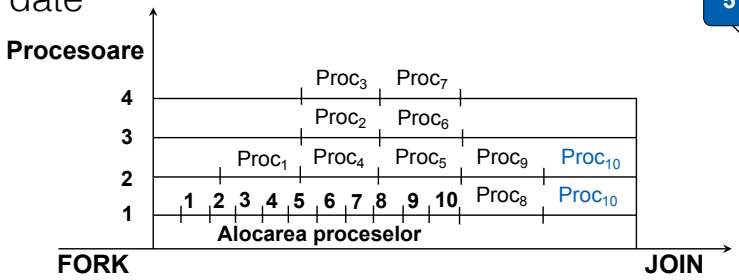




Exemplu cu Dependenta de Date

- Patru procesoare P_1, P_2, P_3, P_4 si 10 procese $Proc_i$ (i = 1…10)
- Aceeasi coada de asteptare
- Presupunem urmatoarea dependenta de date:
- Lucrurile stau altfel: exista procesoare ce asteapta datorita dependentei de date

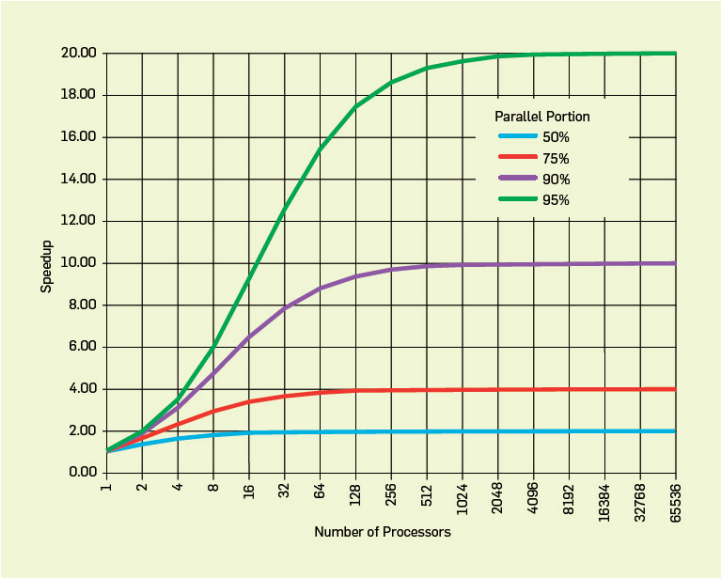




Concluzii Dependenta de Date

- In exemplele prezentate nu apar restrictii de precedenta
- Planificarea FIFO nu e necesar cea mai potrivita
  - Alocare statica
- Alocarea dinamica e mai buna
  - Mai ales cand ∃ procesoare specializate
- Programe de uz general → 10% paralelism
- Programe dedicate → 99+% paralelism
- Dependenta de date → sincronizari in timp
- Eficienta MIMD depinde de:
  - Eficienta algoritmilor de planificare
  - Consumul de timp impus de precedenta de date
- Problema repartizarii pe procesoare → NP completa!

Legea lui Amdahl



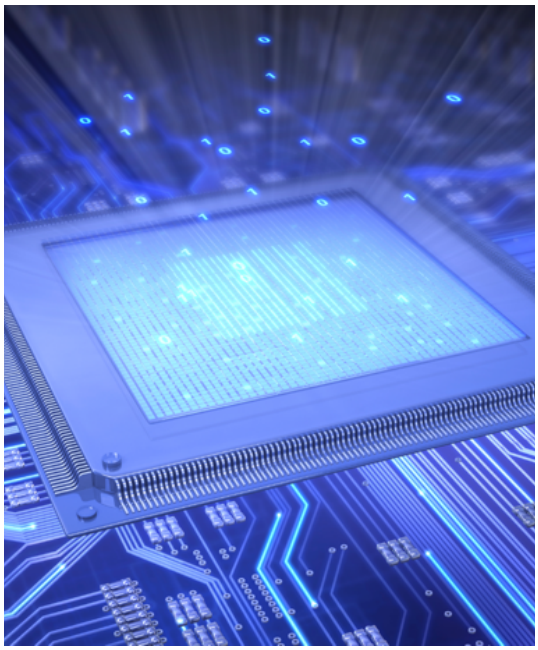
"Keep Them Honest"

Ce am invatat in aces curs?

- De ce avem nevoie de paralelism?
- Care sunt nivelele de prelucrare paralela?
- Clasificarea sistemelor de calcul
- Exemple de utilizare a structurilo: SISS, SIMD, MIMD
- Exemplu cu/fara dependenta de date pe sisteme MIMD
- Limitele scalabilitatii sistemelor paralele de calcul

Slides & Support Material Acknowledgements

Charles M. Lieber 2011
BCA Research 2013
Coding for Speed 2015
IEEE 2016
Applied Materials 2017
Wired & University of Waterloo 2017
Michale Feldman & Intel Roadmap 2017



Thank you for your attention.

emil.slusanschi@cs.pub.ro
cs.pub.ro